

GAN/gan.py

```
1 import numpy as np
2
3 import torch
4 import torch.nn as nn
5 import torchvision.transforms as T
6 import torch.optim as optim
7 from torch import Tensor
8 from torch.utils.data import DataLoader
9
10 NOISE_DIM = 96
11
12 device = torch.device('cuda') if torch.cuda.is_available() else (torch.device('mps') if
13 torch.backends.mps.is_available() else torch.device('cpu'))
14
15 def sample_noise(batch_size: int, dim: int, seed: int | None = None) -> Tensor:
16     """
17     Generate a PyTorch Tensor of uniform random noise.
18
19     Input:
20     - batch_size: Integer giving the batch size of noise to generate.
21     - dim: Integer giving the dimension of noise to generate.
22
23     Output:
24     - A PyTorch Tensor of shape (batch_size, dim) containing uniform
25       random noise in the range (-1, 1).
26     """
27     if seed is not None:
28         torch.manual_seed(seed)
29     return torch.rand(batch_size, dim) * 2 - 1
30
31
32 class Discriminator(nn.Module):
33     def __init__(self):
34         super().__init__()
35
36         #####
37         # TODO: Implement architecture #
38         # #
39         # HINT: nn.Sequential might be helpful. You'll start by calling nn.Flatten()#
40         #####
41         # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
42
43         self.model = nn.Sequential(
44             nn.Flatten(),
45             nn.Linear(784, 256),
46             nn.LeakyReLU(0.01),
47             nn.Linear(256, 256),
```

```

48         nn.LeakyReLU(0.01),
49         nn.Linear(256, 1),
50     )
51
52     # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
53     #####
54     #                                     END OF YOUR CODE                                     #
55     #####
56
57     def forward(self, x):
58         return self.model(x)
59
60
61 class Generator(nn.Module):
62     def __init__(self, noise_dim: int = NOISE_DIM):
63         super().__init__()
64         self.noise_dim = noise_dim
65
66         #####
67         # TODO: Implement architecture                                                    #
68         #                                                                                      #
69         # HINT: nn.Sequential might be helpful.                                          #
70         #####
71         # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
72
73         self.model = nn.Sequential(
74             nn.Linear(self.noise_dim, 1024),
75             nn.ReLU(),
76             nn.Linear(1024, 1024),
77             nn.ReLU(),
78             nn.Linear(1024, 784),
79             nn.Tanh(),
80         )
81
82         # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
83         #####
84         #                                     END OF YOUR CODE                                     #
85         #####
86
87     def forward(self, x):
88         return self.model(x)
89
90
91 def bce_loss(input: Tensor, target: Tensor) -> Tensor:
92     """
93     Numerically stable version of the binary cross-entropy loss function in PyTorch.
94
95     Inputs:
96     - input: PyTorch Tensor of shape (N, 1) giving scores.
97     - target: PyTorch Tensor of shape (N, 1) containing 0 and 1 giving targets.

```

```

98         uses torch.float32, not an integer type.
99
100     Returns:
101     - A PyTorch Tensor containing the mean BCE loss over the minibatch of input data.
102     """
103     bce = nn.BCEWithLogitsLoss()
104     return bce(input, target)
105
106
107 def discriminator_loss(logits_real: Tensor, logits_fake: Tensor) -> Tensor:
108     """
109     Computes the discriminator loss described above.
110
111     Inputs:
112     - logits_real: PyTorch Tensor of shape (N, 1) giving scores for the real data.
113     - logits_fake: PyTorch Tensor of shape (N, 1) giving scores for the fake data.
114
115     Returns:
116     - loss: PyTorch Tensor containing (scalar) the loss for the discriminator.
117
118     HINT: Use torch.ones_like / torch.zeros_like to create label tensors that
119     automatically match the device and dtype of the input.
120     """
121     loss = None
122     # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
123
124     real_labels = torch.ones_like(logits_real)
125     fake_labels = torch.zeros_like(logits_fake)
126     loss = bce_loss(logits_real, real_labels) + bce_loss(logits_fake, fake_labels)
127
128     # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
129     return loss
130
131
132 def generator_loss(logits_fake: Tensor) -> Tensor:
133     """
134     Computes the generator loss described above.
135
136     Inputs:
137     - logits_fake: PyTorch Tensor of shape (N, 1) giving scores for the fake data.
138
139     Returns:
140     - loss: PyTorch Tensor containing the (scalar) loss for the generator.
141
142     HINT: Use torch.ones_like to create label tensors that automatically match
143     the device and dtype of the input.
144     """
145     loss = None
146     # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
147

```

```

148     fake_labels = torch.ones_like(logits_fake)
149     loss = bce_loss(logits_fake, fake_labels)
150
151     # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
152     return loss
153
154
155 def get_optimizer(model: nn.Module) -> torch.optim.Adam:
156     """
157     Construct and return an Adam optimizer for the model with learning rate 1e-3,
158     beta1=0.5, and beta2=0.999.
159     """
160     return optim.Adam(model.parameters(), lr=1e-3, betas=(0.5, 0.999))
161
162
163 def ls_discriminator_loss(scores_real: Tensor, scores_fake: Tensor) -> Tensor:
164     """
165     Compute the Least-Squares GAN loss for the discriminator.
166
167     Inputs:
168     - scores_real: PyTorch Tensor of shape (N, 1) giving scores for the real data.
169     - scores_fake: PyTorch Tensor of shape (N, 1) giving scores for the fake data.
170
171     Outputs:
172     - loss: A PyTorch Tensor containing the loss.
173     """
174     loss = None
175     # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
176
177     loss = 0.5*torch.mean((scores_real-1)**2)+0.5*torch.mean((scores_fake)**2)
178
179     # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
180     return loss
181
182
183 def ls_generator_loss(scores_fake: Tensor) -> Tensor:
184     """
185     Computes the Least-Squares GAN loss for the generator.
186
187     Inputs:
188     - scores_fake: PyTorch Tensor of shape (N, 1) giving scores for the fake data.
189
190     Outputs:
191     - loss: A PyTorch Tensor containing the loss.
192     """
193     loss = None
194     # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
195
196     loss = 0.5*torch.mean((scores_fake-1)**2)
197

```

```

198 # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
199 return loss
200
201
202 class DCDiscriminator(nn.Module):
203     def __init__(self):
204         super().__init__()
205
206         #####
207         # TODO: Implement architecture                                     #
208         #                                                                 #
209         # HINT: nn.Sequential might be helpful.                         #
210         #####
211         # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
212
213         self.model = nn.Sequential(
214             nn.Conv2d(1,32,kernel_size=5,stride=1),
215             nn.LeakyReLU(0.01),
216             nn.MaxPool2d(kernel_size = 2, stride = 2),
217             nn.Conv2d(32,64,kernel_size=5,stride=1),
218             nn.LeakyReLU(0.01),
219             nn.MaxPool2d(kernel_size = 2, stride = 2),
220             nn.Flatten(),
221             nn.Linear(4*4*64,4*4*64),
222             nn.LeakyReLU(0.01),
223             nn.Linear(4*4*64,1),
224         )
225
226         # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
227         #####
228         #                                                                 #
229         #                               END OF YOUR CODE                 #
230         #####
231
232     def forward(self, x):
233         return self.model(x)
234
235 class DCGenerator(nn.Module):
236     def __init__(self, noise_dim: int = NOISE_DIM):
237         super().__init__()
238         self.noise_dim = noise_dim
239
240         #####
241         # TODO: Implement architecture                                     #
242         #                                                                 #
243         # HINT: nn.Sequential might be helpful.                         #
244         #####
245         # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
246
247         self.model = nn.Sequential(

```

```

248         nn.Linear(self.noise_dim,1024),
249         nn.ReLU(),
250         nn.BatchNorm1d(1024),
251         nn.Linear(1024,7*7*128),
252         nn.ReLU(),
253         nn.BatchNorm1d(7*7*128),
254         nn.Unflatten(1, (128, 7, 7)),
255         nn.ConvTranspose2d(128,64, kernel_size=4, stride = 2, padding=1),
256         nn.ReLU(),
257         nn.BatchNorm2d(64),
258         nn.ConvTranspose2d(64,1, kernel_size=4, stride = 2, padding=1),
259         nn.Tanh(),
260         nn.Flatten()
261     )
262
263     # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
264     #####
265     #                                     END OF YOUR CODE                                     #
266     #####
267
268     def forward(self, x):
269         return self.model(x)
270
271
272     def run_a_gan(
273         D: nn.Module,
274         G: nn.Module,
275         D_solver: torch.optim.Optimizer,
276         G_solver: torch.optim.Optimizer,
277         discriminator_loss,
278         generator_loss,
279         loader_train: DataLoader,
280         show_every: int = 250,
281         batch_size: int = 128,
282         noise_size: int = 96,
283         num_epochs: int = 10,
284     ) -> list[np.ndarray]:
285         """
286         Train a GAN!
287
288         Inputs:
289         - D, G: PyTorch models for the discriminator and generator
290         - D_solver, G_solver: torch.optim Optimizers to use for training the
291           discriminator and generator.
292         - discriminator_loss, generator_loss: Functions to use for computing the generator and
293           discriminator loss, respectively.
294         - show_every: Show samples after every show_every iterations.
295         - batch_size: Batch size to use for training.
296         - noise_size: Dimension of the noise to use as input to the generator.
297         - num_epochs: Number of epochs over the training dataset to use for training.

```

```

298     """
299     images = []
300     iter_count = 0
301     for epoch in range(num_epochs):
302         for x, _ in loader_train:
303             D_solver.zero_grad()
304             real_data = x.to(device)
305             logits_real = D(2 * (real_data - 0.5)).to(device)
306
307             g_fake_seed = sample_noise(batch_size, noise_size).to(device)
308             fake_images = G(g_fake_seed).detach()
309             logits_fake = D(fake_images.view(batch_size, 1, 28, 28))
310
311             d_total_error = discriminator_loss(logits_real, logits_fake)
312             d_total_error.backward()
313             D_solver.step()
314
315             G_solver.zero_grad()
316             g_fake_seed = sample_noise(batch_size, noise_size).to(device)
317             fake_images = G(g_fake_seed)
318
319             gen_logits_fake = D(fake_images.view(batch_size, 1, 28, 28))
320             g_error = generator_loss(gen_logits_fake)
321             g_error.backward()
322             G_solver.step()
323
324             if iter_count % show_every == 0:
325                 print(f'Iter: {iter_count}, D: {d_total_error.item():.4f}, G:
326 {g_error.item():.4f}')
327                 imgs_numpy = fake_images.data.cpu().numpy()
328                 images.append(imgs_numpy[0:16])
329
330             iter_count += 1
331
332     return images
333
334 def initialize_weights(m: nn.Module) -> None:
335     if isinstance(m, nn.Linear) or isinstance(m, nn.ConvTranspose2d):
336         nn.init.xavier_uniform_(m.weight.data)
337
338
339 def preprocess_img(x: Tensor) -> Tensor:
340     return 2 * x - 1.0
341
342
343 def deprocess_img(x: Tensor) -> Tensor:
344     return (x + 1.0) / 2.0
345
346

```

```
347 def rel_error(x: np.ndarray, y: np.ndarray) -> float:
348     return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
349
350
351 def count_params(model: nn.Module) -> int:
352     """Count the number of parameters in the model."""
353     return sum(p.numel() for p in model.parameters())
354
```